

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине
«Программное обеспечение мехатронных и робототехнических систем»

Тема лабораторной работы: «Отладка программного обеспечения робототехнических систем с использованием виртуального моделирования»

Шилов М. _____
Студенты группы: КВБО-08-23 Литвинов Л. _____
Тюпалов Т. _____
Преподаватель: Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

Цель работы

Цель работы — изучить принципы виртуального моделирования объектов управления в промышленных системах автоматического управления и освоить создание и использование пользовательских функциональных блоков в Automation Studio.

Задание

Необходимо создать в Automation Studio 4.2 проект без конфигурации оборудования. Проект должен реализовывать виртуальную систему управления с моделью объекта, ПИ-регулятором, сумматором и обратной связью. Передаточная функция объекта управления записывается следующим образом:

$$W = \frac{1}{k_e(T_M s + 1)} \quad (1)$$

Схема построения системы управления представлена на рисунке 1.

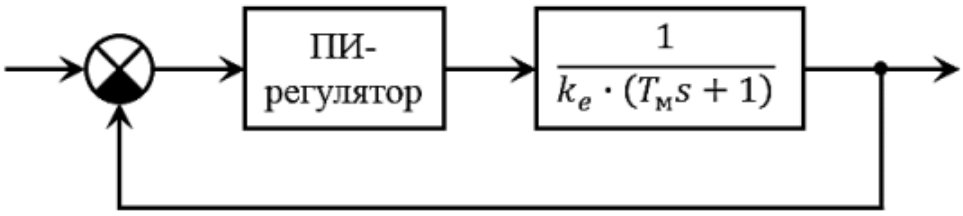


Рисунок 1 – Структура системы управления

Ход работы

В Automation Studio создан проект без конфигурации оборудования. Для работы с виртуальной моделью включена опция «Use Automation Runtime Simulation».

Затем в проект добавлены программа ANSI C Program и библиотека ANSI C Library с именем «MotorControl». В составе библиотеки разработаны три функциональных блока:

- FB_Integrator — интегрирующее звено;
- FB_Motor — модель двигателя постоянного тока;
- FB_Regulator — ПИ-регулятор.

Сначала создан блок FB_Integrator, используемый в двух остальных блоках. Для него заданы вход in, выход out и внутренняя переменная dt. Состав параметров представлен в таблице 1.

Таблица 1 – Параметры функционального блока FB_Integrator

Конфигурация	Имя	Тип данных	Описание
вход	in	REAL	входной сигнал интегратора
выход	out	REAL	выходной сигнал интегратора
внутреннее состояние	dt	REAL	шаг расчета [с]

На рисунке 2 показана структурная схема интегрирующего звена.

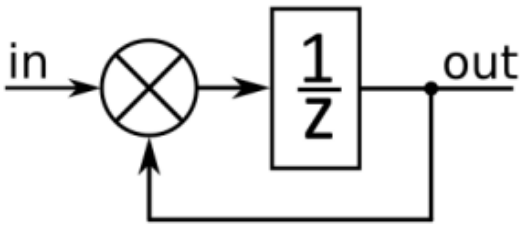


Рисунок 2 – Структурная схема интегратора

Разностное уравнение интегратора получено непосредственно по его структурной схеме:

$$out = out + in * dt \quad (2)$$

С использованием готового интегратора разработан блок FB_Motor. В нём предусмотрены вход u, выходы w и phi и внутренние переменные, перечисленные в таблице 2.

Таблица 2 – Параметры функционального блока FB_Motor

Конфигурация	Имя	Тип данных	Описание
вход	u	REAL	напряжение на входе [В]
выход	w	REAL	частота вращения [об/мин]
выход	phi	REAL	угловое положение [рад]
внутреннее состояние	integrator	FB_Integrator	интегратор
внутреннее состояние	Tm	REAL	электромеханическая постоянная времени [с]
внутреннее состояние	ke	REAL	постоянная ЭДС двигателя [В•мин/об]
внутреннее состояние	dt	REAL	шаг вычисления [с]

На рисунке 3 представлена структура модели объекта управления.

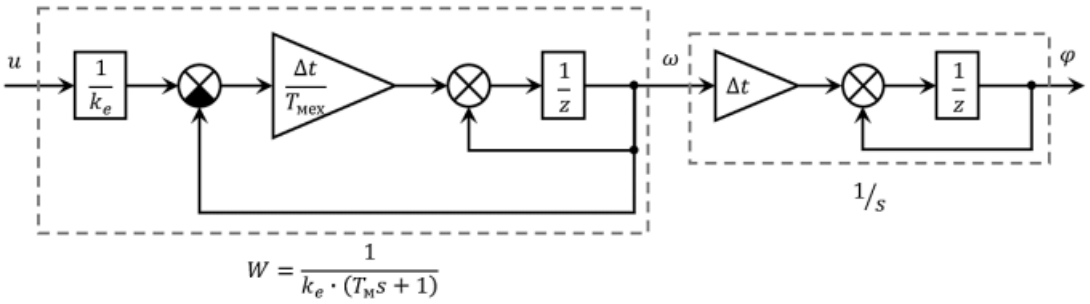


Рисунок 3 – Структура объекта управления

После этого разработан блок FB_Regulator, выполняющий ПИ-регулирование. Его параметры перечислены в таблице 3.

Таблица 3 – Параметры функционального блока FB_Regulator

Конфигурация	Имя	Тип данных	Описание
вход	c	REAL	разность заданной и фактической скорости вала ДПТ [об/мин]
выход	u	REAL	напряжение питания модели ДПТ [В]
внутреннее состояние	k_p	REAL	пропорциональный коэффициент регулятора
внутреннее состояние	k_i	REAL	интегральный коэффициент регулятора
внутреннее состояние	integrator	FB_Integrator	интегратор
внутреннее состояние	iyOld	REAL	хранение предыдущего значения схемы противонакопления
внутреннее состояние	max_abs_value	REAL	предел ограничения сигнала [В]
внутреннее состояние	dt	REAL	шаг вычисления [с]

Схема ПИ-регулятора показана на рисунке 4.

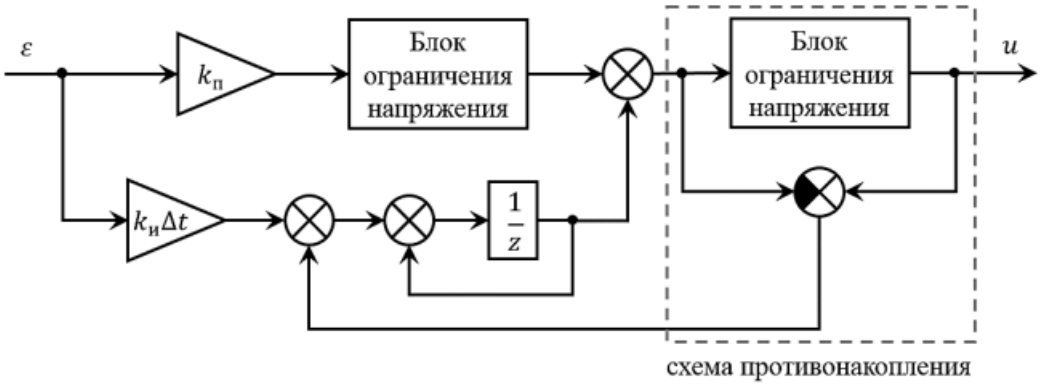


Рисунок 4 – Структура ПИ-регулятора

Коэффициенты регулятора определены методом обратной задачи динамики при $T_ж = 1$:

$$К_п = \frac{k_e * T_m}{T_ж} \quad (3)$$

$$К_и = \frac{k_e}{T_ж} \quad (4)$$

В основной программе заданы переменные fb_controller, fb_motor, Speed и Enable. Их перечень приведён в таблице 4.

Таблица 4 – Переменные основной программы

Имя	Тип данных	Описание
fb_controller	FB_Controller	разность заданной и фактической скорости вала ДПТ [об/мин]
fb_motor	FB_Motor	напряжение питания модели ДПТ [В]
Speed	REAL	заданное значение скорости
Enable	BOOL	интегральный коэффициент регулятора

В разделе инициализации Init заданы постоянные параметры: коэффициенты регулятора, постоянные времени T_m и $T_ж$, постоянная ЭДС k_e, граничные значения и шаги расчёта.

На рисунке 5 показана организация программного обеспечения.

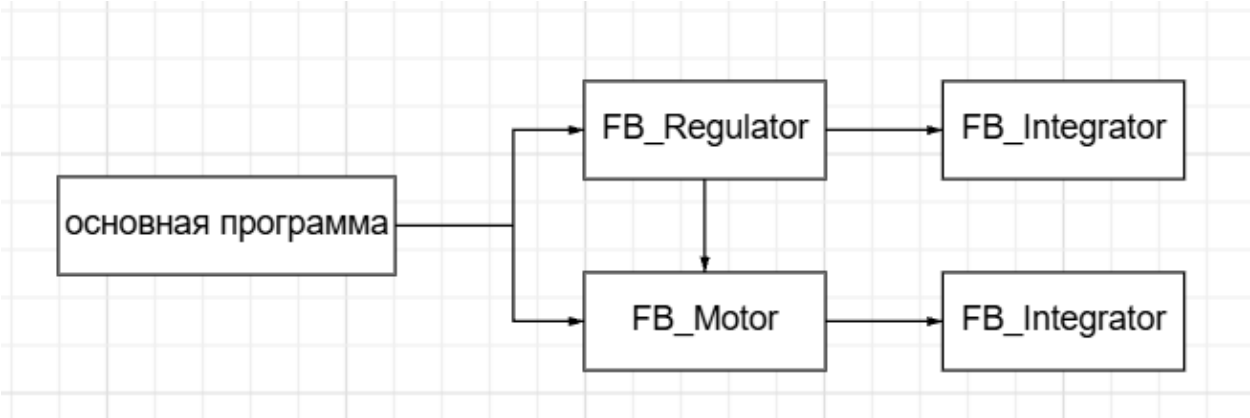


Рисунок 5 – Структура программного обеспечения

Для сопоставления результатов добавлена вторая модель двигателя без регулятора. Ступенчатый входной сигнал задавался с использованием переменной-счётчика counter.

Переходные процессы записаны с помощью средства Trace: в список наблюдения включены нужные переменные, после чего выполнен запуск. Результаты представлены на рисунке 6. Листинг алгоритма управления приведён в приложении А.

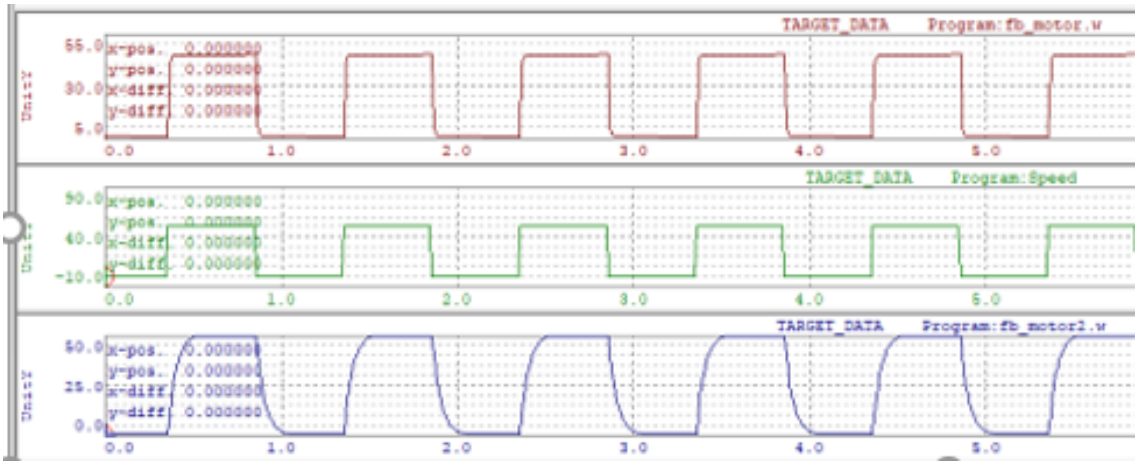


Рисунок 6 – Графики переходных процессов

На рисунке представлены следующие зависимости:

- Верхний график отражает скорость двигателя при работе с настроенным ПИ-регулятором;
- Средний график отображает изменение задания скорости Speed;
- Нижний график отражает скорость двигателя, работающего без регулятора.

Сопоставление верхнего и нижнего графиков показывает, что ПИ-регулятор заметно уменьшает длительность переходного процесса. С регулирующим устройством двигатель достигает заданной скорости в несколько раз быстрее. В рассматриваемом опыте конечное значение скорости осталось прежним, а быстродействие системы повысилось.

Контрольные вопросы

1. В чём заключаются основные преимущества виртуального моделирования исполнительных устройств при отладке программного обеспечения робототехнических и мехатронных систем?

Основные преимущества виртуального моделирования:

- Безопасность - ошибки в программе не приводят к повреждению реального оборудования;
- Сохранение ресурса - испытания не вызывают износ исполнительных устройств;
- Оперативная отладка - параметры модели можно быстро изменять;
- Повторяемость - эксперименты проводятся при одинаковых исходных условиях;

- Возможность выполнять испытания без физического стенда;
- Наглядность - доступны для наблюдения внутренние переменные модели;
- Проверка предельных режимов, включая перегрузки и отказы;
- Сокращение сроков разработки - программную и аппаратную части можно создавать параллельно.

2. На каком этапе разработки проверяется программное обеспечение, выполняемое на реальном контроллере совместно с виртуальной моделью окружающей среды?

Такая проверка проводится на этапе аппаратно-программного моделирования Hardware-in-the-Loop (HIL), который относится к комплексной отладке и испытаниям системы.

Организация проверки:

- Управляющий алгоритм выполняется на физическом контроллере.
- Объект управления и исполнительные устройства представлены виртуальной моделью. Она работает на компьютере или отдельном вычислителе и обменивается данными с контроллером в реальном времени.

На этапе HIL проверяются:

- Временные характеристики работы программы, включая периоды циклов и задержки;
- Корректность обмена через интерфейсы ввода-вывода;
- Обработка сигналов датчиков, формируемых виртуальной моделью;
- Работоспособность и устойчивость системы при помехах и отказах.

3. Какие действия необходимы для реализации закона управления, заданного передаточной функцией с одним входом и одним выходом?

Порядок реализации:

- Записать исходную передаточную функцию W(s);
- Перейти к дискретному описанию с учётом шага расчёта dt и получить форму, пригодную для выполнения на контроллере;
- Составить разностное уравнение, выражающее выход через текущие и предыдущие значения сигналов либо через внутренние состояния;
- Определить структуру реализации: прямую, каноническую, последовательную или параллельную; при необходимости использовать интеграторы и сумматоры;
- Создать функциональный блок, задать его входные и выходные сигналы, коэффициенты и внутренние переменные для хранения состояния;

- Реализовать вычисления по разностному уравнению в теле функционального блока;
- В разделе инициализации установить постоянные времени, коэффициенты усиления, шаг расчёта и начальные значения;
- Включить вызов функционального блока в основную программу;
- Выполнить отладку: получить переходные характеристики, сопоставить их с эталонной моделью и при необходимости уточнить настройки.

4. Как выполняется расчёт регулятора?

Коэффициенты ПИ-регулятора определялись методом обратной задачи динамики. Для программной реализации полученный закон управления переводится в дискретную форму.

Этапы расчёта:

- Записывается передаточная функция объекта управления — двигателя постоянного тока: $W_o(s) = \frac{1}{k_e(T_m * s+1)}$
- Выбирается желаемая передаточная функция замкнутой системы в форме апериодического звена первого порядка: $W_{ж}(s) = \frac{1}{(T_{ж} * s+1)}$
- Из условия соответствия фактической динамики желаемой определяется передаточная функция регулятора:

$$W_p(s) = \frac{(T_m * s + 1)}{(T_{ж} * s * (\frac{1}{k_e}))} = k_e * \frac{(T_m * s + 1)}{T_{ж} * s}$$

- Полученная передаточная функция разделяется на пропорциональную и интегральную части: $W_p(s) = \left(\frac{k_e * T_m}{T_{ж}}\right) * \left(\frac{1}{s}\right) + \left(\frac{k_e}{T_{ж}}\right) * \left(\frac{1}{s}\right) * s$
- Сопоставление с общей записью ПИ-регулятора Wp(s) = Кп + Ки / s позволяет определить коэффициенты:

$$K_{п} = \frac{k_e * T_m}{T_{ж}}$$

$$K_{и} = \frac{k_e}{T_{ж}}$$

5. Что представляет собой функциональный блок и для каких задач он применяется?

Функциональный блок - это программный модуль со следующими свойствами:

- наличие входов, выходов и внутренней памяти;
- выполнение отдельного алгоритма, например регулирования, интегрирования или моделирования двигателя;
- возможность создавать несколько экземпляров для повторного использования;
- сохранение внутренних состояний между последовательными вызовами.

Назначение функциональных блоков:

- Повторное использование - готовый алгоритм применяется в разных частях программы;
- Инкапсуляция - работа с блоком ведётся через его входы и выходы;
- Модульность - сложная программа разделяется на понятные части;
- Удобство отладки - блоки проверяются по отдельности;
- Стандартизация - блоки оформляются как библиотечные элементы Automation Studio;
- Масштабируемость - для новых устройств создаются экземпляры готовых блоков.

Вывод

В работе сравнены переходные процессы двигателя без регулятора и с ПИ-регулятором. Настроенный регулятор обеспечил более быстрый апериодический выход на заданную скорость. Функциональные блоки позволили организовать модульную программу, пригодную для расширения, а средство Trace — оценить переходные процессы и проверить работу системы управления.